# StayEase (Foundation) - Capstone Project for Certification

## Contents

Purpose
1) About Us
2) Problem Statement
3) Roles
4) In-Scope Features
5) User Flows (Happy Path)
7) Architecture
8) Tech Stack
9) Data Model
10) API Outline
11) NFRs & SLAs
12) Testing
13) Engineering PoV
14) Delivery Plan
15) Sample Acceptance Criteria
16) Example Seed Data (SQL)
17) Simple Wireframe
18) Sample User Stories
19) Definition of Ready (DoR) & Done (DoD)
20) Evaluation Rubric (Aligned to Foundation Skills)
22) Stretch Goals (Optional, If Time Permits)
23) Getting Started (Suggested Commands)

## Purpose

Build a hotel room booking web application where users can browse hotels, check room availability for selected dates, and confirm a booking. Hotel managers manage their rooms and bookings. Admins manage the hotel directory. The focus is on foundational full-stack skills - React/Angular, Spring Boot, REST, PostgreSQL, Git, and basic testing - aligned with the Foundation skill matrix.

## 1) About Us

At StayEase, we believe finding and booking a hotel room should take minutes. This Foundation version delivers a clean hotel search, room availability check, and booking experience - no payment gateway, no real-time map, just a complete stay flow.

## 2) Problem Statement

Travellers need a simple way to find available hotel rooms for their dates, view room details and pricing, and confirm a booking without making phone calls.

Goal: Deliver a minimal yet complete hotel room booking flow demonstrating full-stack fundamentals.

## 3) Roles

1. Guest/User - Can browse hotels, check availability, book and cancel rooms.
2. Hotel Manager - Can manage rooms for their hotel and view upcoming bookings.
3. Admin - Can manage the hotel directory.

Note: Role stored as String field: String role; // "GUEST" | "MANAGER" | "ADMIN"

### Role Flag / Field Approach

- User entity: String role; (defaults to GUEST on registration)
- MANAGER and ADMIN accounts are seeded by Admin.
- Backend: check role in controller before allowing hotel/room CRUD.
- Frontend: Manager dashboard visible only to MANAGER; Admin panel to ADMIN.

### Backend Concept

- Backend: check role in controller before allowing hotel/room CRUD.
- Frontend: Manager dashboard visible only to MANAGER; Admin panel to ADMIN.

### Frontend Concept

- Backend: check role in controller before allowing hotel/room CRUD.
- Frontend: Manager dashboard visible only to MANAGER; Admin panel to ADMIN.

## 4) In-Scope Features

### Authentication (Basic)

- Register, Login, Logout.

- Simple password hashing (BCrypt).

### Hotel Search & Browse

- Search form: City, Check-in Date, Check-out Date.
- Results: hotels in the city with name, star rating, starting price, cover image.
- Hotel detail page with description, amenities list (text), and room types.

### Room Availability & Booking

- Available rooms for selected dates shown with type (Single/Double/Suite), price per night, and max occupancy.
- Room is considered available if no overlapping booking exists for those dates.
- Confirm booking -> Booking ID generated, total price = nights x pricePerNight.
- My Stays page: list of bookings with hotel, room, dates, total price, status.
- Cancel a booking (room becomes available again).

### Manager Dashboard

- CRUD for rooms (roomNumber, roomType, pricePerNight, maxOccupancy, description, imageUrl, isActive).
- View upcoming bookings for their hotel.

### Admin - Hotel Management

- CRUD for hotels (name, city, starRating, description, coverImageUrl, managerId).

---

## 5) User Flows (Happy Path)

1. Search: Guest enters City + Check-in/Check-out dates -> sees hotel list.
2. Detail: Clicks hotel -> sees description + available rooms for selected dates.
3. Auth: If not logged in at booking -> prompted to register/login.
4. Confirm: Clicks Book on a room -> reviews total price -> confirms -> sees Booking ID.
5. My Stays: Views all bookings -> can cancel upcoming stays.
6. Manager: Logs in -> adds/edits rooms -> views upcoming bookings.
7. Admin: Logs in -> adds/edits hotels in the directory.

---

## 7) Architecture

- Frontend: React (Hooks + Context API) or Angular (Services + Components). Routing enabled.
- Backend: Spring Boot (REST Controllers, Services, Spring Data JPA).
- Database: PostgreSQL (Dev may use H2 for quick start).
- Auth: Stateless JWT or simple session cookie (choose one; document trade-offs).
- State: Selected dates passed as query params; availability computed on backend.
- Deployment: Localhost

---

## 8) Tech Stack

### Frontend

- React + Vite/CRA (Routing, Hooks, Context) or Angular (Routing, Services)
- HTML5, CSS3, JavaScript (ES6), TypeScript

### Backend

- Java 17+, Spring Boot, Spring Web, Spring Data JPA, Validation
- JUnit5, Mockito

### Database

- PostgreSQL, JPA/Hibernate
- H2 for local

### Tools & Practices

- Git Basics (branching, PRs), Postman/Swagger, Agile ceremonies
- AI Pair Programming: GitHub Copilot/ChatGPT for boilerplate and explanations

---

## 9) Data Model

### Entities & Relationships

- Hotel (1) -- (M) Room
- User (1) -- (M) Booking [as Guest]
- Room (1) -- (M) Booking
- Booking stores: checkInDate, checkOutDate, totalPrice, status, bookingRef

Note: Availability query: SELECT r FROM Room r WHERE r.hotel.id = :hotelId AND r.id NOT IN (SELECT b.room.id FROM Booking b WHERE b.status != 'CANCELLED' AND b.checkInDate < :checkOut AND b.checkOutDate > :checkIn)

---

## 10) API Outline

### Auth

- POST /api/auth/register - {email, password, name} -> 201
- POST /api/auth/login -> 200 {token}
- POST /api/auth/logout -> 204

### Hotels

- GET /api/hotels?city= - list hotels in city
- GET /api/hotels/{id}/rooms?checkIn=&checkOut;= - available rooms
- POST /api/hotels (admin) - create
- PUT /api/hotels/{id} (admin)
- DELETE /api/hotels/{id} (admin)

### Rooms

- POST /api/hotels/{id}/rooms (manager) - create room
- PUT /api/rooms/{id} (manager)
- DELETE /api/rooms/{id} (manager)
- PATCH /api/rooms/{id}/status (manager) - toggle isActive

### Bookings

- POST /api/bookings - {roomId, checkInDate, checkOutDate} -> creates booking
- GET /api/bookings/mine - guest's bookings

- PUT /api/bookings/{id}/cancel - cancel booking

### Error Format (Consistent)

```
{
"timestamp": "2025-09-12T12:00:00Z",
"path": "/api/...",
"error": "VALIDATION_ERROR",
"message": "Field is required"
}
```

---

## 11) NFRs & SLAs

- Performance:
  - API response <= 2 s under light load (1-5 concurrent users locally).
  - UI renders initial content within 3 s on localhost.
- Security:
  - Passwords hashed (BCrypt); JWT/session token stored securely.
  - Basic input validation & server-side checks.
- Scalability/Concurrency:
  - Test with 5-10 concurrent requests locally. Simulate using Postman Runner or multiple browser tabs. No JMeter or load-testing tools required at this level.
- Error Handling:
  - User-friendly error toasts/messages on UI; structured error JSON on API.
- Logging:
  - Console logs; simple request/response logs (no external logging infra).
- Documentation:
  - README.md with setup steps, seed data, and run commands.
  - API contracts documented with Swagger/OpenAPI.

---

## 12) Testing

- Backend Unit Tests
  - Services & Repos with JUnit5 + Mockito
  - Target coverage: >=60% (method-level)
- API Testing
  - Manual via Postman or Swagger.
- Frontend
  - Optional unit tests for 1-2 key components (e.g., Booking form).
- No automated regression/e2e required.

---

## 13) Engineering PoV

- Code Quality:
  - Follows SOLID basics where practical (e.g., service layer, DTOs).
  - No critical lint errors (ESLint/TSLint).
- Branching:
  - main (stable), feature/* branches; PR reviews by peers/mentor.

- Coverage:
  - Aim 60-70% backend, frontend minimal.
- Dependencies:
  - Keep list short; beware of transitive vulnerabilities.
- Docs:
  - API endpoints documented; Postman collection included.

---

## 14) Delivery Plan (3 Sprints, 1-2 weeks each)

### Sprint 1 - Foundations & Hotel Browse

- Scaffolding (FE + BE), DB setup, Git workflow
- Hotel entity, Room entity; GET /hotels?city= and GET /hotels/{id}/rooms
- Frontend: Home + Hotel Search + Hotel List + Hotel Detail with rooms
- Seed data loading (hotels, rooms)

### Sprint 2 - Auth, Availability & Booking

- Register/Login; protect booking and manager routes
- Date-based availability query; booking creation with total price calculation
- My Stays page; cancel booking; validation + error handling

### Sprint 3 - Manager Dashboard & Admin

- Manager: Room CRUD + view upcoming bookings for their hotel
- Admin: Hotel CRUD
- Polish: README, Postman collection, unit tests >=60%

---

## 15) Sample Acceptance Criteria

### Hotel Search

- Given I search city='Goa' with valid dates, I see at least 2 seeded hotels.
- Each hotel shows name, star rating, and starting price.

### Room Availability

- Given I select dates that overlap an existing booking, that room does NOT appear as available.
- After cancelling a booking, the room reappears as available for those dates.

### Booking

- After confirming, I see a Booking ID and the correct total price (nights x rate).
- My Stays page shows the booking with correct hotel, room type, dates, and status.

### Manager Dashboard

- Only manager of that hotel can manage its rooms via /manager/rooms.
- Manager sees a list of all upcoming bookings for their hotel.

---

## 16) Example Seed Data (SQL)

### SQL

```
INSERT INTO users(id,email,password_hash,role,name,created_at)
VALUES (gen_random_uuid(),'admin@stayease.com','$2a$10$hash','ADMIN','Admin',now()),
(gen_random_uuid(),'mgr1@stayease.com','$2a$10$hash','MANAGER','Hotel Mgr',now()),
(gen_random_uuid(),'guest1@stayease.com','$2a$10$hash','GUEST','Rohit Verma',now());

INSERT INTO hotel(id,name,city,star_rating,description,cover_image_url,manager_id,created_at)
VALUES
(gen_random_uuid(),'Sunset Palms','Goa',4,'Beachfront resort with pool',
'https://picsum.photos/seed/hotel1/400',
(SELECT id FROM users WHERE email='mgr1@stayease.com'),now()),
(gen_random_uuid(),'City Inn','Goa',3,'Budget-friendly city centre hotel',
'https://picsum.photos/seed/hotel2/400',
(SELECT id FROM users WHERE email='mgr1@stayease.com'),now());

INSERT INTO room(id,hotel_id,room_number,room_type,price_per_night,max_occupancy,is_active,created_at)
VALUES
(gen_random_uuid(),(SELECT id FROM hotel WHERE name='Sunset Palms'),'101','Double',3500.00,2,true,now()),
(gen_random_uuid(),(SELECT id FROM hotel WHERE name='Sunset Palms'),'201','Suite',7000.00,3,true,now()),
(gen_random_uuid(),(SELECT id FROM hotel WHERE name='City Inn'),'101','Single',1500.00,1,true,now());
```

---

## 17) Simple Wireframe

- Home: Search panel (City / Check-in Date / Check-out Date / [Search Hotels])
- Hotel List: Cards - [Cover] Hotel Name - Stars - City - Price from X/night - [View Rooms]
- Hotel Detail: [Cover] Name - Description - Amenities - Available Rooms list (Type/Price/Max) - [Book]
- My Stays: Table (Hotel / Room Type / Check-in / Check-out / Total / Status) - [Cancel]
- Manager Dashboard: My Rooms table (Number/Type/Price/Active toggle) + Upcoming Bookings table

---

## 18) Sample User Stories

- As a guest, I want to search hotels by city and dates so I can find an available room.
- As a guest, I want to see room details and price so I can choose within my budget.
- As a guest, I want to confirm a booking so I can secure my room for the dates.
- As a guest, I want to cancel a booking so I can free the room if plans change.
- As a manager, I want to add rooms so guests can discover and book them.

---

## 19) Definition of Ready (DoR) & Done (DoD)

### DoR

- User story has clear acceptance criteria.
- API/DB changes identified (if any).
- Mock/UX reference exists (even a sketch).

### DoD

- Code compiles; unit tests for backend services pass.
- Manual tests completed (Postman + UI).
- No critical ESLint/TS issues; meaningful README update.
- Peer review completed; merged to main.

---

## 20) Evaluation Rubric (Aligned to Foundation Skills)

| Area | What We Check | Weight |
|---|---|---|
| Frontend (React/Angular) | Routing, hooks/services, component communication, state handling (Context/Service), basic TypeScript usage | 25% |
| Backend (Spring Boot) | Clean REST controllers, services, JPA entities & relationships, validation, basic error handling | 25% |
| Common Skills | Core Java correctness, SQL/JPA usage, SOLID basics, Git hygiene, HTML/CSS semantics | 20% |
| Testing | JUnit/Mockito for services/repos; Postman collection; target >=60% coverage | 15% |
| Generative AI Usage | Appropriate use of Copilot/ChatGPT for boilerplate, explanations, refactoring (cite prompts in README) | 10% |
| Documentation & Readme | Setup steps, seed data, API list, constraints & trade-offs | 5% |

---

## 22) Stretch Goals (Optional, If Time Permits)

- Star rating filter on the hotel search results.
- Number of nights and total price displayed dynamically as dates change.
- Booking confirmation page with a printable summary.

---

## 23) Getting Started (Suggested Commands)

### Backend

- Java 17+, Spring Boot.
- Profiles: dev (H2), prod (Postgres).

### Frontend

- React: npm create vite@latest -ui -- --template react-ts
- Angular: ng new -ui --routing --style=scss

### Run Order

1. Start DB (H2 or Postgres) 2. Run Spring Boot 3. Seed Data 4. Run UI

---